

Transformers and Large Language Models (LLMs): Fundamentals

Introduction

Transformers and Large Language Models (LLMs) have become foundational technologies in modern artificial intelligence. They power applications such as chatbots, coding assistants, translation systems, search tools, summarization software, and many other AI products. Although LLMs can appear remarkably intelligent, their underlying operation is based on a set of mathematical and computational techniques that can be understood systematically.

The Transformer architecture, introduced in 2017, changed natural language processing by providing an efficient way to model relationships between words and other pieces of information in a sequence. Unlike earlier recurrent neural networks, Transformers can process many parts of a sequence in parallel while using attention mechanisms to determine which pieces of information are most relevant to one another.

An LLM is essentially a very large neural network trained on enormous quantities of text. Most modern LLMs are based on the Transformer architecture. Understanding the fundamentals therefore requires examining several interconnected concepts: tokens, embeddings, attention, Transformer layers, positional information, training, inference, parameters, and the limitations of language models.

1. What Is a Transformer?

A Transformer is a neural network architecture designed originally for processing sequential data, particularly natural language. The architecture was introduced in the influential 2017 research paper *Attention Is All You Need*.

Before Transformers, language-processing systems frequently used Recurrent Neural Networks (RNNs) and Long Short-Term Memory networks (LSTMs). These models processed sequences step by step. For example, when reading a sentence, an RNN would process one word, update its internal state, process the next word, and continue until reaching the end.

This sequential approach created difficulties. Long sequences could be computationally expensive, and information from the beginning of a sentence could become difficult to preserve by the time the model reached the end.

Transformers addressed these limitations through the attention mechanism. Instead of processing information strictly one element at a time, a Transformer can examine relationships between many tokens simultaneously. This makes training much more parallelizable and allows models to learn long-range relationships more effectively.

2. Tokens: How Text Becomes Numbers

Neural networks cannot directly process ordinary written language. Text must first be converted into numerical representations.

The first major step is tokenization. A tokenizer divides text into smaller units called tokens. A token may represent a complete word, part of a word, punctuation, or another frequently occurring sequence of characters.

For example, a sentence such as:

"Transformers are powerful."

might be divided into several tokens representing "Transformers", "are", "powerful", and punctuation. The exact tokenization depends on the tokenizer and vocabulary used by the model.

Each token is assigned an integer ID. The original text is therefore transformed into a sequence of numbers. These token IDs are then converted into vectors through an embedding layer.

Tokenization is important because it determines how efficiently a model can represent language. If tokens are too large, the vocabulary can become enormous. If tokens are too small, sequences become unnecessarily long. Modern tokenization techniques attempt to balance these factors.

3. Embeddings

An embedding is a numerical vector representing a token. Instead of representing a word simply as an integer such as 5, a neural network converts the token ID into a high-dimensional vector containing learned numerical values.

For example, a token might be represented conceptually by a vector such as:

[0.21, -0.47, 0.83, 0.15, ...]

Real models use vectors with hundreds or thousands of dimensions.

During training, the model learns useful representations of tokens. Tokens that occur in similar contexts often develop related representations. Importantly, embeddings do not simply store dictionary definitions. Their values are learned from statistical patterns in training data.

Embeddings provide the numerical foundation on which the Transformer performs further computation.

4. The Attention Mechanism

Attention is the central idea behind the Transformer architecture.

Suppose we have the sentence:

"The animal didn't cross the road because it was tired."

To understand what "it" refers to, a language model needs to examine relationships between different parts of the sentence. Attention allows the model to assign different levels of importance to other tokens when processing a particular token.

The standard Transformer attention mechanism uses three concepts: queries, keys, and values.

A query represents what a token is looking for. A key represents information that can be matched against that query. A value contains the information that is ultimately retrieved.

The attention calculation can be simplified conceptually as:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

Here, Q represents queries, K represents keys, V represents values, and d_k is the dimensionality of the key vectors.

The query-key calculation produces similarity scores. Softmax converts these scores into weights. The weighted values are then combined to create a new representation.

This mechanism allows each token to consider other relevant tokens when constructing its contextual representation.

5. Self-Attention

Self-attention means that the tokens within the same sequence attend to one another.

For example, in:

"The student opened the book because she needed information."

the representation of "she" can use information from "student" and other tokens to establish context.

Self-attention is powerful because relationships do not have to be local. A token can potentially attend to another token much farther away in the sequence.

However, standard self-attention has a computational cost that grows significantly as sequence length increases. This is one reason why processing extremely long contexts can be expensive.

6. Multi-Head Attention

Transformers generally use multi-head attention rather than a single attention mechanism.

Multiple attention heads allow the model to examine different types of relationships simultaneously. One head may learn patterns associated with grammatical relationships, another may focus on semantic relationships, and another may capture connections between distant parts of a sequence.

The outputs of the attention heads are combined and transformed into the representation passed to subsequent layers.

The exact behaviors learned by individual attention heads are not always simple or interpretable, but the overall mechanism provides the network with multiple ways to analyze contextual relationships.

7. Positional Information

Attention itself does not inherently understand the order of tokens. The sentences:

"Dog bites man."

and

"Man bites dog."

contain the same words but have very different meanings.

Therefore, Transformers need positional information. Early Transformer architectures used positional encodings based on mathematical functions. Modern models may use alternative techniques such as learned position representations or rotary positional embeddings.

Positional information allows the model to distinguish between tokens based not only on their identity but also on their location and relationship within the sequence.

8. Transformer Layers

A Transformer is built from repeated layers. Although architectures vary, a typical Transformer layer contains an attention component followed by a feed-forward neural network, along with residual connections and normalization.

The attention mechanism allows tokens to exchange contextual information. The feed-forward network then performs additional transformations on each token representation.

Residual connections help information flow through many layers and make deep networks easier to train. Normalization techniques help stabilize the computations.

A large language model may contain dozens or even hundreds of such layers. Each layer progressively transforms the representations, allowing the model to learn increasingly sophisticated patterns.

9. Parameters

Parameters are the numerical values learned during model training. They include the weights of embedding layers, attention mechanisms, feed-forward networks, and other components.

When people describe a model as having billions of parameters, they are referring to the approximate number of learned numerical values in the network.

A larger parameter count generally provides greater representational capacity, but parameter count alone does not determine model quality. Training data, architecture, optimization methods, data quality, context length, and post-training techniques are also extremely important.

The goal of training is to adjust these parameters so that the model becomes increasingly good at its learning objective.

10. How LLMs Are Trained

Most modern generative LLMs begin with a process commonly called pretraining.

During pretraining, the model is exposed to a very large collection of text. A common objective is next-token prediction. Given a sequence of tokens, the model attempts to predict the token that comes next.

For example:

"The capital of France is"

might lead the model to assign a high probability to "Paris".

The model initially makes poor predictions because its parameters are randomly initialized or otherwise untrained. During training, the predicted probability distribution is compared with the actual next token. A loss value measures the error.

An optimization algorithm, usually based on gradient descent and backpropagation, calculates how the parameters should be changed to reduce the loss.

This process is repeated over enormous numbers of examples.

Importantly, the model does not memorize a simple collection of rules in the traditional sense. Training causes statistical patterns to become encoded across its parameters. The resulting network can then generalize those patterns to new sequences.

11. From Pretraining to Useful Assistants

Pretraining creates a powerful general-purpose language model, but it does not automatically make the model a helpful conversational assistant.

Additional training processes can be used to improve instruction following, safety, helpfulness, and response quality. These processes are often collectively referred to as post-training.

One approach involves supervised fine-tuning, where the model is trained on examples demonstrating desirable responses to instructions.

Another family of techniques uses human or AI feedback to encourage preferred behaviors. The exact methods vary between organizations and models.

The result is a model that is not merely optimized to predict text but is additionally adapted to respond to user instructions in useful ways.

12. Inference: How an LLM Generates Text

Inference is the process of using a trained model to generate a response.

When a user enters a prompt, it is tokenized and passed through the Transformer. The model produces probabilities for possible next tokens.

Suppose the model has generated:

"The sky is"

It might assign probabilities such as:

-
- blue: 0.70
-
-

clear: 0.10
•
•
cloudy: 0.08
•
•
bright: 0.05
•
•
other tokens: 0.07
•

The model selects a token according to its decoding strategy. That token is added to the sequence, and the process repeats.

This continues until the model reaches an appropriate stopping condition.

Although the model generates one token at a time during inference, the Transformer architecture allows the training process to be highly parallelized.

13. Temperature and Sampling

The probability distribution produced by an LLM can be modified during generation.

Temperature is one commonly discussed parameter. Lower temperatures make the distribution more concentrated, generally producing more predictable outputs. Higher temperatures make less-probable choices more likely, which can increase variation.

Other decoding techniques include top-k and top-p sampling. These methods restrict the candidate tokens considered during generation.

However, sampling does not turn the model into a random idea generator. The underlying probability distribution still comes from the learned model.

14. Context Windows

An LLM processes a sequence of tokens within a context window. The context can include a user's prompt, previous conversation messages, documents, instructions, and other information provided to the model.

A larger context window allows a model to process longer inputs, but longer contexts can increase computational requirements.

It is also important to distinguish between context and long-term memory. Information included in the current context can influence the model's response, but that does not necessarily mean the model permanently learns the information.

15. Why LLMs Can Appear Intelligent

LLMs can produce surprisingly sophisticated responses because language contains enormous amounts of information about the world, reasoning patterns, social interactions, programming, mathematics, and human communication.

During training, the model learns statistical relationships across this information. Its many layers can transform representations in complex ways.

The model does not necessarily possess human-like consciousness or understanding. Nevertheless, large-scale neural networks can develop capabilities that emerge from training at sufficient scale.

This is one of the most important lessons about modern AI: relatively simple learning objectives, when combined with huge datasets, substantial computation, and capable architectures, can produce surprisingly powerful systems.

16. Limitations of LLMs

Despite their capabilities, LLMs have important limitations.

First, they can generate incorrect information. This is often called hallucination. A model may produce an answer that sounds confident and plausible even when it is factually wrong.

Second, an LLM's knowledge depends on its training and the information available through its context or connected tools. A model without access to current information may not know about recent events.

Third, models can reproduce biases present in their training data. Careful dataset construction, evaluation, and post-training are therefore important.

Fourth, LLMs can struggle with certain forms of precise reasoning. They can perform impressive mathematical and logical tasks, but reliability can vary depending on the problem and the methods used.

Finally, running large models requires significant computational resources. Training and inference can involve large numbers of GPUs or specialized accelerators, substantial electricity consumption, and complex software infrastructure.

17. Transformers Beyond Text

Although Transformers became famous through natural language processing, the architecture is not limited to text.

Transformers are now used in computer vision, audio processing, multimodal systems, biological modeling, robotics, and other areas.

The underlying idea is broadly useful: represent information as tokens or token-like units and learn relationships between them using attention.

This flexibility is one reason the Transformer architecture has become one of the most influential developments in modern machine learning.

Conclusion

Transformers and LLMs represent a major shift in artificial intelligence. At the heart of the technology is the Transformer architecture, particularly its attention mechanism. Text is converted into tokens, tokens are represented as embeddings, attention allows the model to incorporate contextual relationships, and repeated Transformer layers progressively transform those representations.

Large Language Models are trained primarily through large-scale optimization on massive datasets. By learning to predict tokens and subsequently undergoing additional training, these systems can become capable of generating coherent text, following instructions, writing code, summarizing documents, answering questions, and performing many other tasks.

Understanding the fundamentals of Transformers helps explain both the capabilities and limitations of modern AI. LLMs are not simply databases of sentences. They are large learned mathematical systems whose parameters encode patterns acquired during training. During inference, they use those learned patterns to calculate probabilities for possible next tokens and generate responses sequentially.

The field continues to evolve rapidly. Future systems are likely to combine increasingly capable Transformer-based models with external tools, retrieval systems, multimodal inputs, specialized reasoning methods, and improved training techniques. Nevertheless, the fundamental concepts of tokens, embeddings, attention, parameters, training, and inference remain essential for understanding how modern LLMs work.

For anyone beginning their study of generative AI, these concepts provide the foundation for progressing toward more advanced subjects such as fine-tuning, retrieval-augmented generation, model quantization, inference optimization, multimodal models, AI agents, and Transformer architecture design.